

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Narappagari sravya	Reg. No.:	192472192
Section / Batch:	B.E CSE-AI-2024	Date:	16-07-2026

Code of Conduct

I Narappagari sravya(192472192) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: sravya

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions**3. Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANNEXURE A – CODING CHALLENGE

Section 1: Intelligent Email and Password Validator using Regular Expressions

Problem Statement

A software company is developing a secure user registration system to allow new users to create accounts safely. Before a user account is created, the system must verify that all entered details satisfy predefined validation rules. This helps prevent invalid data entry and improves system security.

Design and implement a **Python-based validation system** using **Regular Expressions (Regex)** to verify the following user details.

Functional Requirements

The application should perform the following tasks:

- Accept an email address from the user.
- Validate whether the email follows the correct format.
- Accept a password from the user.
- Verify whether the password satisfies all security requirements.
- Accept a mobile number from the user.
- Validate whether the mobile number is valid.
- Display appropriate validation messages.
- Display the overall registration status.

CODE:

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

# Email Validation
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'

# Password Validation
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!])[A-Za-z\d@#%&!]{8,}$'

# Mobile Validation
mobile_pattern = r'^[6-9]\d{9}$'

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
```

```
print("Weak Password")
```

```
if re.fullmatch(mobile_pattern, mobile):  
    print("Valid Mobile Number")  
else:  
    print("Invalid Mobile Number")
```

OUTPUT:

Enter Email: sravya@gmail.com

Enter Password: 123456

Enter Mobile Number: 9876453423

Valid Email

Weak Password

Valid Mobile Number

```
>>> ===== RESTART: C:/Users/sravy/AppData/Local/Programs/Python/Python310/3.1.py =====  
Enter Email: sravya@gmail.com  
Enter Password: 123456  
Enter Mobile Number: 9876453423  
Valid Email  
Weak Password  
Valid Mobile Number  
>>>
```

Section 2: Design a Finite State Automata (FSA) Simulator

Problem Statement

Finite State Automata (FSA) are mathematical models used in compiler design, lexical analysis, pattern matching, and natural language processing. They recognize whether a given input string belongs to a particular language.

Develop a **Python-based Deterministic Finite Automata (DFA) Simulator** that accepts a DFA description and simulates its execution for different input strings.

Functional Requirements

The application should:

- Accept DFA states.
- Accept the input alphabet.
- Accept the transition table.
- Accept the initial state.
- Accept one or more final states.
- Read multiple input strings.
- Simulate transitions between states.
- Display every state transition.
- Display whether the string is accepted or rejected.

CODE:

```
# DFA for strings ending with "ab"
```

```
transition = {  
    ('q0', 'a'): 'q1',  
    ('q0', 'b'): 'q0',  
    ('q1', 'a'): 'q1',
```

```

    ('q1', 'b'): 'q2',
    ('q2', 'a'): 'q1',
    ('q2', 'b'): 'q0'
}

start_state = 'q0'
final_state = 'q2'

n = int(input("Enter number of input strings: "))

for i in range(n):
    string = input("Enter string: ")

    state = start_state
    path = [state]

    for ch in string:
        if (state, ch) in transition:
            state = transition[(state, ch)]
            path.append(state)
        else:
            state = None
            break

    print("Transition Path:")
    print(" -> ".join(path))

    if state == final_state:
        print("Accepted")
    else:
        print("Rejected")

```

OUTPUT:

```

Enter number of input strings: 2
Enter string: abaab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter string: abb
Transition Path:
q0 -> q1 -> q2 -> q0
Rejected

```

```

>>> ===== RESTART: C:/Users/sravy/Downloads/NATURAL LANGUAGE PROCESSING/3.2.py =====
Enter number of input strings: 2
Enter string: abaab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter string: abb
Transition Path:
q0 -> q1 -> q2 -> q0
Rejected
>>>

```

Section 3: Smart Pattern Matching Engine using Regular Expressions

Problem Statement

Pattern matching is one of the most important applications of Regular Expressions in Natural Language Processing. Search engines, social media platforms, and text processing applications use pattern matching to identify specific information from large amounts of text.

Develop a **Python-based Smart Pattern Matching Engine** using Regular Expressions to search different types of patterns from a given text.

Functional Requirements

The application should:

- Read a paragraph or multiple lines of text.
- Display a menu for different search operations.
- Search words using Regular Expressions.
- Search prefixes.
- Search suffixes.
- Search dates.
- Search phone numbers.
- Search hashtags.
- Search mentions (@username).
- Display all matching patterns.

CODE:

```
import re
```

```
text = """
```

```
Meeting on 12/09/2026
```

```
Call 9876543210
```

```
#NLP
```

```
@OpenAI
```

```
natural language processing
```

```
"""
```

```
while True:
```

```
    print("\n1.Search Date")
```

```
    print("2.Search Phone Number")
```



```

print("3.Search Hashtag")
print("4.Search Mention")
print("5.Search Prefix")
print("6.Search Suffix")
print("7.Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    print(re.findall(r'\b\d{2}\d{2}\d{4}\b', text))

elif choice == 2:
    print(re.findall(r'\b[6-9]\d{9}\b', text))

elif choice == 3:
    print(re.findall(r'#\w+', text))

elif choice == 4:
    print(re.findall(r'@\w+', text))

elif choice == 5:
    prefix = input("Enter Prefix: ")
    print(re.findall(r'\b' + prefix + r'\w*', text, re.IGNORECASE))

elif choice == 6:
    suffix = input("Enter Suffix: ")
    print(re.findall(r'\b\w*' + suffix + r'\b', text, re.IGNORECASE))

elif choice == 7:
    print("Program Ended")
    break

else:
    print("Invalid Choice")

```

Expected Output

```

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Exit
Enter your choice: 1
['12/09/2026']

1.Search Date

```

- 2.Search Phone Number
- 3.Search Hashtag
- 4.Search Mention
- 5.Search Prefix
- 6.Search Suffix
- 7.Exit

Enter your choice: 2

['9876543210']

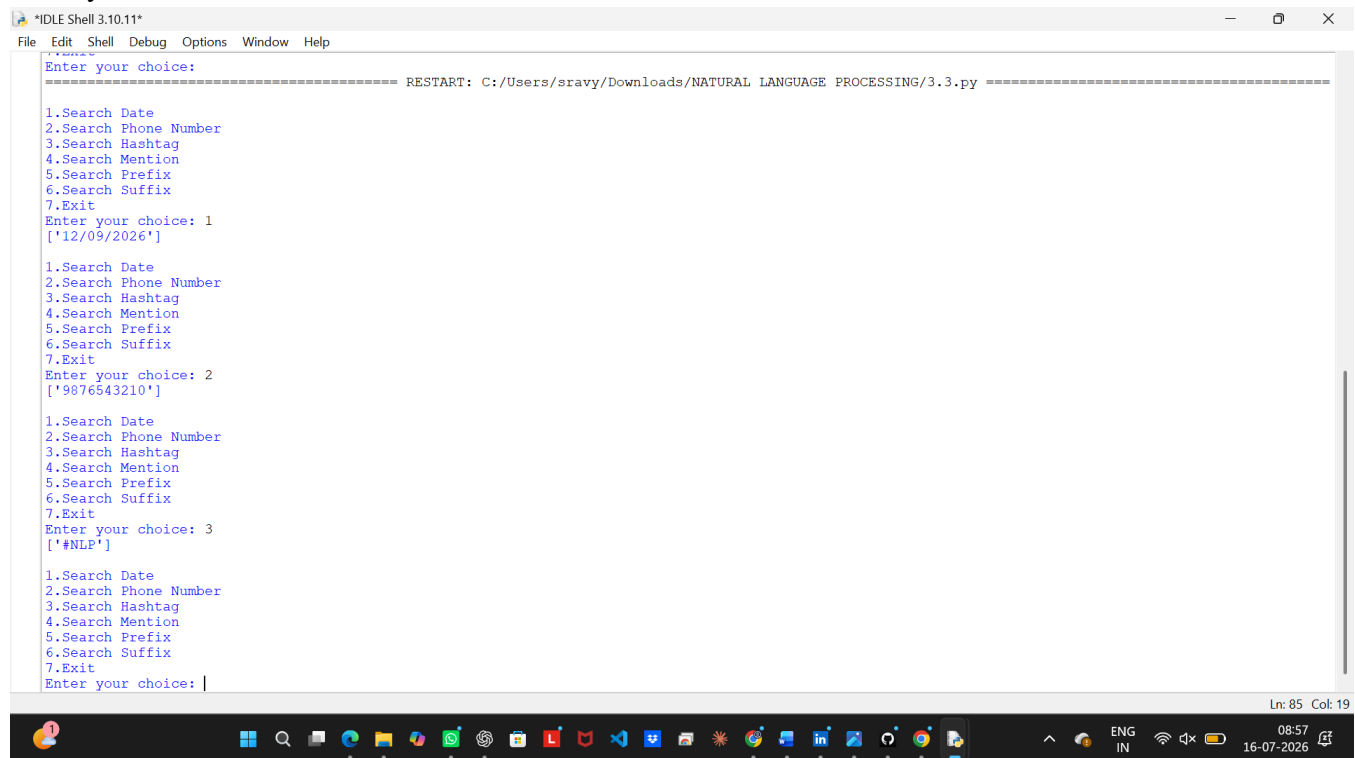
- 1.Search Date
- 2.Search Phone Number
- 3.Search Hashtag
- 4.Search Mention
- 5.Search Prefix
- 6.Search Suffix
- 7.Exit

Enter your choice: 3

['#NLP']

- 1.Search Date
- 2.Search Phone Number
- 3.Search Hashtag
- 4.Search Mention
- 5.Search Prefix
- 6.Search Suffix
- 7.Exit

Enter your choice:



```
*IDLE Shell 3.10.11*
File Edit Shell Debug Options Window Help
Enter your choice:
===== RESTART: C:/Users/sravy/Downloads/NATURAL LANGUAGE PROCESSING/3.3.py =====
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Exit
Enter your choice: 1
['12/09/2026']

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Exit
Enter your choice: 2
['9876543210']

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Exit
Enter your choice: 3
['#NLP']

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Exit
Enter your choice: |
```